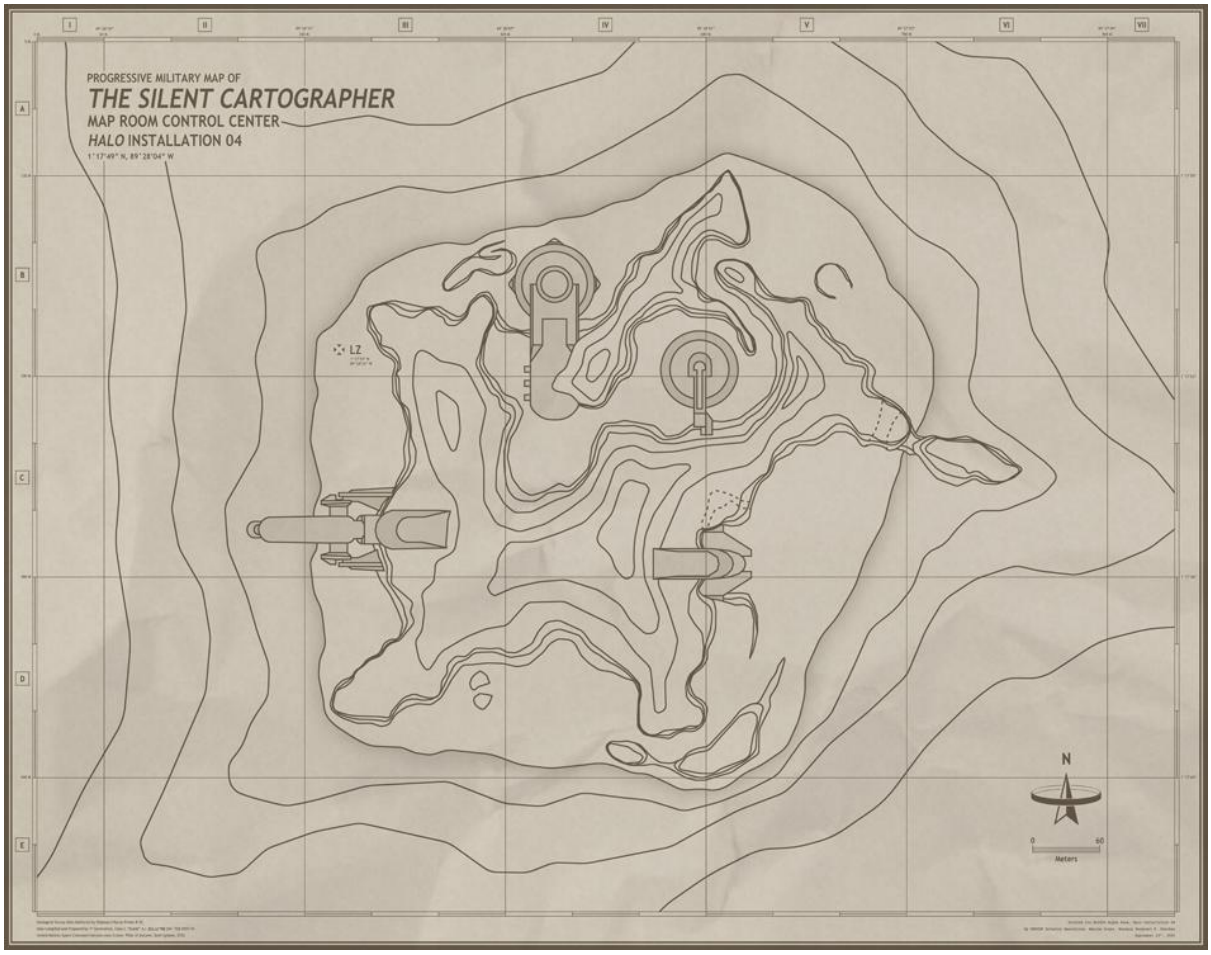


# Silent Cartographer: Maze Navigation Project

COSC 4368 AI Spring 2026
Prof: Dr. Raunak Sarbajna. TA: Mert Saritac
Last Updated: Feb 3 6pm

---

## 1. Adaptive Maze Navigation using Reinforcement Learning and Greedy Search Algorithms

### 1.1 Project Description

Students will develop an intelligent agent capable of navigating unknown 64x64 mazes containing hazards using a combination of reinforcement learning and classical search algorithms. The agent must learn to explore efficiently, build an internal map representation, and devise optimal navigation strategies despite imperfect information and environmental hazards.

### 1.2 Learning Outcomes

Upon completion, students will be able to:

- Design and implement hybrid AI systems combining classical and modern approaches.
- Formulate navigation problems as Markov Decision Processes.
- Develop exploration strategies for unknown environments.
- Implement and compare multiple pathfinding algorithms.
- Build robust agents that handle uncertainty and partial observability.
- Analyse and communicate technical results effectively.

### 1.3 Team Requirements

- **Team Size:** 5 students per group *maximum*
- **Team Formation Deadline:** February 6, 2026
- **Project Duration:** April 20, 2026

### 1.4 Team Checkpoints

- **Checking In** Every three weeks.
    - Load Maze into python with and without hazard.
    - Do naïve solve.
    - Implement hazard solution.
    - Implement full solution.

---

# 2. ENVIRONMENT SPECIFICATIONS

## 2.1 Maze Structure

#### 2.1.1 Dimensions

- Grid size: **64 × 64 cells**
- Total cells: 4,096 positions
- Coordinate system: (0,0) at top-left, (63,63) at bottom-right

#### 2.1.2 Cell Types

1. **Empty cells** - Safe, passable
2. **Walls** - Impassable barriers
3. **Start position** - Agent spawning location (marked ‘S’)
4. **Goal/Exit** - Target destination (marked ‘G’)
5. **Death pits** - Instant death, agent respawns at start (marked ‘P’)
6. **Teleport pads** - Transport agent to another location (marked ‘T’)
7. **Confusion pads** – Inverts Agent control (marked ‘C’)

#### 2.1.3 Maze Properties

- Guaranteed to have at least one valid path from start to goal.

- Walls form connected barriers (no single-cell isolated walls).
- Hazard density: 5-10% of navigable cells.
- Multiple valid solution paths may exist.

## 2.2 Training and Testing Mazes

### 2.2.1 Training Maze

- Provided at project start, week of February 9.
- Fixed seed for reproducibility.
- Students may run unlimited episodes on training maze.
- Full solution path exists.

### 2.2.2 Testing Maze

- Provided on April 6.
- Different layout from training maze.
- Similar statistical properties (size, hazard density, complexity).
- Students get limited attempts on test maze.
- Used for final evaluation.

---

# 3. AGENT SPECIFICATIONS

## 3.1 Agent Capabilities

### 3.1.1 Perception

- **No visual/sensor input** of unexplored areas.
- Knows current position coordinates after each turn.
- Receives feedback on action outcomes (see Section 3.3).
- Cannot see through walls or detect hazards at distance.

### 3.1.2 Memory

- **Perfect memory** - unlimited storage capacity.
- Can maintain any data structures (maps, graphs, statistics).
- Memory persists across episodes during training.
- Can store complete history of all exploration.

### 3.1.3 Actions

Agent can execute the following atomic actions:

1. **MOVE_UP** - Attempt to move north (-1 to y-coordinate)
2. **MOVE_DOWN** - Attempt to move south (+1 to y-coordinate)
3. **MOVE_LEFT** - Attempt to move west (-1 to x-coordinate)
4. **MOVE_RIGHT** - Attempt to move east (+1 to x-coordinate)

5. **WAIT** - Remain in current position (useful for planning)

## 3.2 Turn Structure

### 3.2.1 Action Submission

- Each turn, agent submits **1 to 5 actions.**
- Actions execute sequentially in order submitted.
- If agent dies or reaches goal, remaining actions ignored.
- No time limit per turn (within reasonable bounds).

### 3.2.2 Turn Execution

```
1. Agent submits action list: [action1, action2, ..., action5]
2. Environment processes each action sequentially
3. Environment returns TurnResult after all actions complete
4. Agent updates internal state and plans next turn
5. Repeat until episode ends
```

## 3.3 Turn Feedback

After each turn, the agent receives a `TurnResult` object containing:

```
class TurnResult:
    wall_hits: int              # Number of wall collisions this turn (0-5)
    current_position: Tuple[int, int]  # Agent's (x, y) coordinates
    is_dead: bool               # True if agent stepped on death pit
    is_goal_reached: bool       # True if agent reached exit
    teleported: bool            # True if teleport was triggered this turn
    actions_executed: int       # Number of actions completed before
death/goal
```

### 3.3.1 Feedback Details

**wall_hits:**

- Increments by 1 for each action that attempts invalid movement.
- Hitting a wall does NOT change agent position.
- Agent remains in position before the invalid move.

**current_position:**

- Agent's coordinates after all actions executed.
- If dead: position of death pit.
- If teleported: destination coordinates.
- Always accurate and truthful.

**is_confused:**

- Set to `True` if agent stepped on confusion pit during this turn.
- Every single movement for the rest of the turn and for the following turn is inverted.
- The inversion process is:

    - MOVE_UP <-> MOVE_DOWN
    - MOVE_LEFT <-> MOVE_RIGHT

**is_dead:**

- Set to `True` if agent stepped on death pit during this turn.
- Agent immediately respawns at start position next turn.
- Episode does NOT end on death.
- Death counter increments for scoring purposes.

**is_goal_reached:**

- Set to `True` if agent reaches goal cell.
- Episode ends immediately.
- Triggers success metrics calculation.

**teleported:**

- Set to `True` if agent stepped on teleport pad.
- Agent does not know teleport destination in advance.
- Teleport destinations are deterministic per pad.
- current_position shows post-teleport location.

**actions_executed:**

- Number of actions completed (1-5).
- Less than submitted if death or goal reached mid-turn.
- Useful for debugging and understanding turn flow.

---

# 4. HAZARD MECHANICS

## 4.1 Death Pits

### 4.1.1 Behaviour

- Agent dies instantly upon entering pit cell.
- Agent respawns at start position on next turn.
- Episode continues (does not end).
- Death counter increments.

### 4.1.2 Detection

- No visual indication before stepping on pit.
- Agent must infer pit locations from death events.
- Perfect memory allows building hazard map.

### 4.1.3 Scoring Impact

- Each death incurs penalty in final scoring.
- Excessive deaths indicate poor exploration strategy.

## 4.2 Teleport Pads

### 4.2.1 Behavior

- Stepping on teleport pad instantly moves agent.
- Destination is deterministic for each specific teleport pad.
- Teleport destinations are navigable cells (not walls/pits).
- Agent can teleport multiple times in one turn if chained.

### 4.2.2 Detection

- `teleported` flag indicates teleport occurred.
- Agent must map teleport sources and destinations.
- Teleport graph may be complex (multi-hop, one-way).

### 4.2.3 Strategic Considerations

- Teleports may provide shortcuts or may create complex exploration challenges. You should be able to be avoided them if you have mapped the maze properly.

## 4.3 Walls

### 4.3.1 Behaviour

- Block all movement attempts.
- Agent position unchanged when hitting wall.
- `wall_hits` counter increments.

### 4.3.2 Detection

- No prior knowledge of wall locations.
- Must be discovered through exploration.
- `wall_hits > 0` indicates obstacle encountered.

## 4.4 Confusion Traps

### 4.4.1 Behaviour

- Agent has confusion status applied on touching the trapped cell.
- For the rest of the turn, and for the following turn all movements are reversed.
    - Top and Down invert, Right and Left Invert

### 4.4.2 Detection

- Agent is Aware that it is currently confused and has to adjust behaviour accordingly.

- Agent must check `is_confused` status every turn to become aware of hitting the trap and marking the location.

#### 4.1.3 Scoring Impact

- Is completely neutral, has no penalty.
- The number of times confused must be logged.

---

# 5. EPISODE STRUCTURE

## 5.1 Episode Initialization

```
1. Agent spawns at start position (S)
2. All memory from previous episodes retained (during training)
3. Turn counter resets to 0
4. Death counter resets to 0
```

## 5.2 Episode Execution

```
WHILE episode_active:
    1. Agent plans and submits actions (1-5 actions)
    2. Environment executes actions sequentially
    3. Environment returns TurnResult
    4. Agent updates internal state/memory
    5. Turn counter increments

    IF is_goal_reached:
        Episode ends with SUCCESS
    IF turn_counter >= MAX_TURNS:
        Episode ends with TIMEOUT
```

## 5.3 Episode Termination Conditions

#### 5.3.1 Success

- Agent reaches goal cell (is_goal_reached = True).
- Episode marked as successful.
- Performance metrics calculated.

#### 5.3.2 Timeout

- Agent exceeds maximum turn limit.
- Episode marked as failed.
- Partial progress may still provide learning signal.

#### 5.3.3 Manual Termination

- Agent can voluntarily end episode (if you *know* that you have reached a location that cannot be escaped from).

### 5.4 Episode Limits

**Training Phase:**

- Maximum turns per episode: **10,000 turns**
- Unlimited episodes allowed.
- No time limit on total training.

**Testing Phase:**

- Maximum turns per episode: **10,000 turns**
- Maximum test episodes: **5 episodes**
- Best performance of 5 episodes used for grading.

---

# 6. API SPECIFICATION

## 6.1 Core Interface

```
from typing import List, Tuple
from enum import Enum

class Action(Enum):
    MOVE_UP = 0
    MOVE_DOWN = 1
    MOVE_LEFT = 2
    MOVE_RIGHT = 3
    WAIT = 4

class TurnResult:
    def __init__(self):
        self.wall_hits: int = 0
        self.current_position: Tuple[int, int] = (0, 0)
        self.is_dead: bool = False
        self.is_confused: bool = False
        self.is_goal_reached: bool = False
        self.teleported: bool = False
        self.actions_executed: int = 0

class MazeEnvironment:
    def __init__(self, maze_id: str):
        """
        Initialize maze environment

        Args:
            maze_id: 'training' or 'testing'
        """
        pass

    def reset(self) -> Tuple[int, int]:
        """
        Reset environment for new episode

        Returns:
            Starting position coordinates
```

```
        """
        pass

    def step(self, actions: List[Action]) -> TurnResult:
        """
        Execute a turn with given actions

        Args:
            actions: List of 1-5 Action objects

        Returns:
            TurnResult with feedback

        Raises:
            ValueError: If actions list empty or >5 actions
        """
        pass

    def get_episode_stats(self) -> dict:
        """
        Get statistics for current episode

        Returns:
            Dictionary containing:
            - turns_taken: int
            - deaths: int
            - confused: int
            - cells_explored: int
            - goal_reached: bool
        """
        pass

class Agent:
    """
    Base class for student implementations
    Students must implement this interface
    """

    def __init__(self):
        """
        Initialize agent with empty memory
        """
        self.memory = {}  # Students can structure as needed

    def plan_turn(self, last_result: TurnResult) -> List[Action]:
        """
        Plan next set of actions based on last turn result

        Args:
            last_result: Feedback from previous turn
                         (None on first turn of episode)

        Returns:
            List of 1-5 actions to execute
        """
        raise NotImplementedError("Students must implement this method")

    def reset_episode(self):
        """
        Called at start of new episode
        Students can reset episode-specific state
```

```
        Memory can be retained for learning
        """
        pass
```

## 6.2 Evaluation Interface

```
class Evaluator:
    """
    Provided by instructors for grading
    """

    def evaluate_agent(self,
                       agent: Agent,
                       maze_id: str,
                       num_episodes: int = 5) -> dict:
        """
        Run agent on specified maze

        Args:
            agent: Student's agent implementation
            maze_id: 'training' or 'testing'
            num_episodes: Number of evaluation episodes

        Returns:
            Performance metrics dictionary
        """
        pass

    def get_metrics(self) -> dict:
        """
        Returns:
            Dictionary with:
            - success_rate: float (0-1)
            - avg_turns: float
            - avg_deaths: float
            - avg_path_length: float
            - exploration_efficiency: float
        """
        pass
```

## 6.3 Visualization Interface (Optional Support)

```
class Visualizer:
    """
    Utility for visualizing agent's internal map
    """

    def visualize_map(self, agent_memory: dict) -> None:
        """Display agent's known map"""
        pass

    def replay_episode(self, episode_data: dict) -> None:
        """Animate episode execution"""
        pass

    def plot_learning_curves(self, training_data: dict) -> None:
        """Plot training progress"""
        pass
```

---

# 7. DELIVERABLES

### 7.1 Final Report (5-10 pages)

### 7.2.2 Final Presentation (15 minutes + 5 min Q&A)

### 7.2.3 Code Submission Method:

- GitHub repository (public, add members as collaborators)
- Or ZIP file upload to MS Teams
- Include trained model weights
- Tag final submission as release v1.0

---

# 8. EVALUATION METRICS

## 8.1 Performance Metrics

### 8.1.1 Primary Metrics

**Success Rate:**

```
success_rate = (successful_episodes / total_episodes) × 100%
```

- Successful episode: agent reaches goal within turn limit.
- Calculated over 5 test episodes.

**Average Path Length:**

```
avg_path_length = mean(path_lengths for successful episodes)
```

- Path length: number of cells visited (including duplicates)
- Excludes teleport jumps from count.
- Lower is better.

**Average Turns to Solution:**

```
avg_turns = mean(turns_taken for successful episodes)
```

- Number of turns required to reach goal.
- Lower is better.

**Death Rate:**

```
death_rate = total_deaths / total_turns
```

- Calculated over all test episodes.
- *Lower* is better.

- Indicates exploration safety.

#### 8.1.2 Secondary Metrics

**Exploration Efficiency:**

```
exploration_efficiency = unique_cells_discovered / total_cells_visited
```

- Measures redundancy in exploration
- Higher is better (max 1.0)

**Map Completeness:**

```
map_completeness = known_cells / total_navigable_cells
```

- Percentage of maze discovered.
- Higher indicates thorough exploration.

**Replanning Efficiency:**

- Average time to replan after discovering new obstacles.
- Relevant for search-based approaches

**Learning Efficiency (Training only):**

- Episodes until convergence.
- Sample efficiency of RL algorithm.

### 8.2 Bonus Opportunities (up to 10 extra points)

**Competition Rankings:**

- **1st place (fastest solution):** 5 bonus points
- **1st place (most efficient path):** 5 bonus points
- **1st place (safest exploration):** 3 bonus points

**Technical Excellence:**

- **Novel visualzation:** up to 5 points
- **Open-source toolkit release:** up to 5 points

**Note:** Maximum total bonus is 10 points

---

# 9. ACADEMIC INTEGRITY

### 9.1 Collaboration Policy

**Permitted:**

- Discussion of general approaches with other teams.
- Sharing knowledge of algorithms from literature.
- Using course-provided code and APIs.
- Consulting published papers and textbooks.

**Prohibited:**

- Sharing code between teams.
- Copying code from online sources without attribution.
- Using solutions from previous years.
- Having others write code for your team.
- Unauthorized use of external debugging/coding assistance tools.

## 10.2 Citation Requirements

**Code Attribution:**

- Clearly mark and cite any adapted code.
- Include source URL and license information.
- Describe modifications made.

**Written Work:**

- Cite all sources using IEEE or ACM format
- Minimum 10 academic references in final report
- Properly attribute ideas and algorithms

## 10.3 Use of AI Coding Assistants

**Permitted:**

- Using tools like GitHub Copilot for boilerplate code
- Using AI for debugging assistance
- Using AI for documentation generation

**Required:**

- Disclose AI tool usage in README.
- Understand and be able to explain all code.
- Significant algorithmic design must be original.

**Violations:**

- Any violation of academic integrity policy results in zero for the project.
- Serious violations reported to university administration.
- All team members held responsible for integrity violations.

---

# 10. FREQUENTLY ASKED QUESTIONS

### Q1: Can we use pre-trained models?

**A:** No pre-trained models on maze navigation tasks are permitted. You may use pre-trained feature extractors (e.g., CNN backbones) if you can justify their relevance, but the navigation policy must be learned from scratch on the provided mazes.

### Q2: What happens if our agent doesn't solve the test maze?

**A:** Partial credit is awarded based on progress. You can still earn points for implementation quality, checkpoints, and report. However, reaching the goal at least once is important for full credit.

### Q3: Can we train on mazes we generate ourselves?

**A:** Yes, you may create additional training mazes for supplementary training, but you must document this and your agent must perform well on the official training maze.

### Q4: How are teleport destinations determined?

**A:** Each teleport pad has a deterministic destination. The same teleport always leads to the same location. Teleports are one-way (teleport pads don't exist at destinations).

### Q5: Can we modify the environment or API?

**A:** No, the provided environment and API must be used as-is. Your agent must work with the standard interface. Modifications will result in penalties.

### Q6: What if team members aren't contributing equally?

**A:** Teams can include a contribution statement in their final report. In cases of serious imbalance, contact me or your TA **early**. We can facilitate mediation or, in extreme cases, adjust individual grades.

### Q7: Can we use the test maze for debugging before final submission?

**A:** Yes, but you're limited to 5 complete runs. Choose wisely when to test. Excessive querying may result in penalties.

### Q8: Are walls always aligned to the grid?

**A:** Yes, everything is grid-aligned. Each cell is entirely one type (empty, wall, pit, teleport, etc.).

### Q9: Can the agent sense adjacent cells?

**A:** Only through attempted movement. The agent gets no sensor data. It must try to move to discover walls.

### Q10: How is "path length" measured?

**A:** Total cells visited, including revisits. If you visit cell (5,5) three times, it counts as 3 toward path length.

---

# 11. APPENDIX

## 11.1 Sample Code Snippets

### Basic Agent Template

```
from typing import List
from environment import Action, TurnResult, MazeEnvironment

class MyAgent:
    def __init__(self):
        self.map = {}  # Store discovered information
        self.current_pos = None

    def plan_turn(self, last_result: TurnResult) -> List[Action]:
        """
        Your planning logic here
        """
        if last_result is not None:
            self.update_map(last_result)

        # Example: simple exploration
        actions = [Action.MOVE_RIGHT]
        return actions

    def update_map(self, result: TurnResult):
        """Update internal map based on turn result"""
        self.current_pos = result.current_position
        # Add your mapping logic here
```

### Running an Episode

```
env = MazeEnvironment('training')
agent = MyAgent()

pos = env.reset()
done = False
turn_count = 0

last_result = None

while not done and turn_count < 10000:
    actions = agent.plan_turn(last_result)
    last_result = env.step(actions)

    if last_result.is_goal_reached:
```

```
        print(f"Success in {turn_count} turns!")
        done = True

    turn_count += 1

stats = env.get_episode_stats()
print(f"Final stats: {stats}")
```

## 11.2 Maze Format Specification

The maze is internally represented as a 64×64 integer array:

- `0` = Empty/navigable cell
- `1` = Wall
- `2` = Start position
- `3` = Goal position
- `4` = Death pit
- `5` = Teleport pad

Remember that your agents do NOT have access to this array. They must discover it through exploration.

## 11.3 Example Test Cases

Teams should test their agents on various scenarios:

- Simple corridor navigation
- Dense wall mazes
- Multiple teleport chains
- High pit density areas
- Long optimal paths

Sample test mazes (smaller sizes) will be provided for validation.

## 11.4 Performance Benchmarks

**Baseline Random Agent:**

- Success rate: ~5%
- Average turns (if successful): ~8000
- Death rate: ~0.15

**Expected Student Performance:**

- Success rate: >80%
- Average turns: <1000
- Death rate: <0.05

**Stretch Goals:**

- Success rate: >95%

- Average turns: <500
- Death rate: <0.01

**Document Version:** 1.2
**Last Updated:** January 29, 2026
**Subject to minor clarifications; major changes will be announced on Teams and in Class**